Design and Implementation of an Image Cartoonifier using FPGA

23ECE383 VLSI Design Laboratory

B. Tech. ECE

Batch: B1

2025-2026 Odd Semester

CB.EN.U4ECE23114	DeenaThayalan K G
CB.EN.U4ECE23118	Gunda Rohith
CB.EN.U4ECE23130	Raghuram Surapaneni

Faculty In-charges: Dr. Abhishek Raj, Dr. Prabhu E.

Signature of the Faculty:

Department of Electronics and Communication Engineering

Amrita School of Engineering

Amrita Vishwa Vidyapeetham

Amritanagar, Coimbatore – 641112

TABLE OF CONTENTS

1. Objective:

   The aim of the project is to design and implement an Image Cartoonifier for 4 different modes: Color, Black and white, Grayscale, and Sober Filter using FPGA (Basys 3). This system aims to showcase the hardware – software integration of a UART for receiving the pixels generated by a python script and a VGA controller for real - time display of the cartoonified image on the monitor.

2. Software hardware used:

   ~ **Software used:** Xilinx Vivado Design Suite Vivado 2020.1, Python IDLE 3.11
   ~ **Hardware used:** Basys 3 FPGA development board - equipped with Xilinx Artix-7 FPGA device

3. Overview of the system:

   a. Clock Generation and System Core

   The clock_gen_basys3.v module is the foundation of the system's timing. It takes the 100 MHz clock from the Basys3 board and uses a Mixed-Mode Clock Manager (MMCM) to generate two key clock signals:

   - A 25 MHz clk_pix clock for the VGA controller, which is the standard pixel clock for 640x480 resolution.
   - A buffered 100 MHz clk_sys for the main logic, including the UART receiver, debouncers, and processing pipeline. The module also provides a locked signal to indicate that the MMCM has stabilized, which is used as a reset signal for other modules.

   b. User Inputs and Control Logic

   The main.v module handles user interaction via switches and buttons on the Basys3 board.

   - Mode Selection: A pair of switches (sw[1:0]) is used to select one of four image processing modes:
     - Black & White: Converts the image to a simple black-and-white format based on a threshold of 128.
     - Grayscale: Quantizes the 8-bit grayscale data into 8 distinct levels for a more refined grayscale display.
     - Color: Maps the 8-bit grayscale input data to a false-color palette with 16 distinct color gradients.
     - Sobel Edge Detection: Processes the image to highlight edges.

   - Reset: A debounce module (debounce_onepulse.v) is used to filter out noise from the btn_clear button press, ensuring a single, clean pulse. This pulse triggers a state machine that writes zeros to the entire framebuffer, effectively clearing the display.

```
### Switches
- **SW[1:0]**: Mode selection
  - `00`: Black & White mode
  - `01`: Enhanced Grayscale (8 levels)
  - `10`: Enhanced Color (16 levels)
  - `11`: Sobel Edge Detection
```

c. Image Data Flow

The system's pipeline manages the flow of image data from reception to display.

- UART Reception: The uart_rx.v module receives 8-bit pixel data serially from a PC. It operates at a baud rate of 1,250,000. Once a complete byte is received, the rx_valid signal is asserted and the rx_data is made available.
- Pixel Processing: The system's main logic processes each incoming pixel (rx_data) based on the selected mode.
    - For the black & white, grayscale, and false-color modes, the input pixel is directly processed and stored as processed_pixel.
    - For the Sobel mode, the raw pixel data is passed to the sobel3x3_stream.v module for edge detection.
- Sobel Edge Detection: The sobel3x3_stream.v module is a dedicated hardware block for this task. It uses two internal line buffers (implemented with block RAM) to store previous rows of pixel data, enabling it to operate on a 3x3 pixel window. It calculates the horizontal (gx) and vertical (gy) gradients, finds the magnitude, and compares it against a fixed threshold to determine if a pixel is an edge.

d. Framebuffer and VGA Display

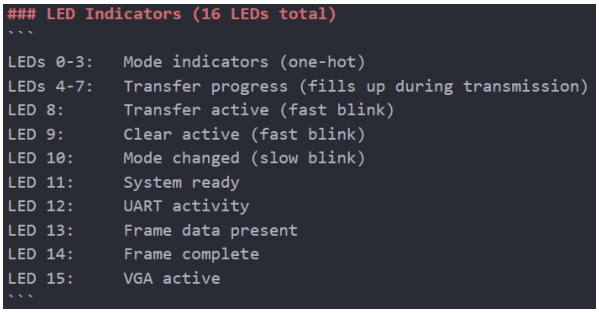
The processed image data is stored and then read out for display.

- Framebuffer: The system uses a simple dual-port block RAM as a framebuffer to store the processed image. One port is used to write data from the processing pipeline, while the other is used to read data for the VGA display. The write address is calculated from the x_sys and y_sys coordinates of the incoming pixel, while the read address is generated by the VGA controller.
- VGA Controller: The vga_640x480.v module generates the necessary timing signals (Hsync and Vsync) for a 640x480 VGA display. It also provides the current pixel coordinates (vga_x and vga_y) and an active signal to indicate when the display is in the visible area.
- VGA Output: Based on the current mode, the main.v module translates the 8-bit data read from the framebuffer (fb_rd_data) into 4-bit red, green, and blue (vgaRed, vgaGreen, vgaBlue) signals for the VGA output. The color mapping logic differs for each mode, providing the appropriate grayscale, false color, or black-and-white output.

e. LED Indicators

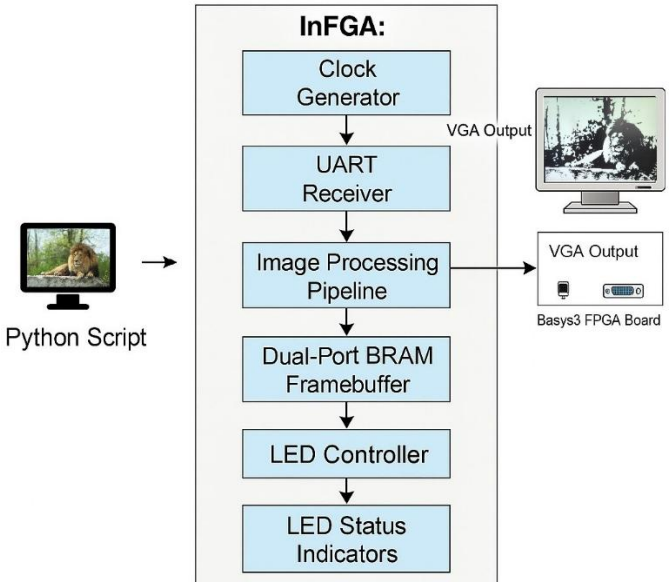
LEDs on the Basys3 board provide visual feedback on the system's status and operation.

- Mode: LEDs 0-3 act as a one-hot indicator for the currently selected mode.
- Transfer Progress: LEDs 4-7 indicate the percentage of the image that has been received from the UART, providing a visual progress bar.
- Status: Other LEDs indicate various statuses, such as whether a new frame transfer is active, a clear operation is in progress, the mode has been changed, the PLL is locked, or the VGA display is active.



```
### LED Indicators (16 LEDs total)
```
LEDs 0-3:   Mode indicators (one-hot)
LEDs 4-7:   Transfer progress (fills up during transmission)
LED 8:   Transfer active (fast blink)
LED 9:   Clear active (fast blink)
LED 10:   Mode changed (slow blink)
LED 11:   System ready
LED 12:   UART activity
LED 13:   Frame data present
LED 14:   Frame complete
LED 15:   VGA active
```
```

4. Block diagram:



5. Introduction:

This project implements a real-time image processing and display system on a Basys3 FPGA, focusing on hardware-accelerated image manipulation. The system processes incoming 8-bit grayscale image data received via a high-speed UART interface from a PC. The core of the design is a processing pipeline that applies various image filters, based on the user's selection via switches. These modes include Black & White, enhanced Grayscale, false Color mapping, and a hardware-accelerated Sobel edge detection algorithm. The processed image is then stored in a dedicated on-chip framebuffer.

A dedicated VGA controller concurrently reads from the framebuffer to continuously refresh a 640x480 VGA display, enabling real-time visualization of the processed image. The VGA controller generates the necessary timing signals **(Hsync and Vsync)** and provides pixel coordinates for the framebuffer read port. The system leverages the FPGA's parallel processing for efficient image operations, such as the 3x3 Sobel filter, while using the UART for flexible data transfer. This design demonstrates a complete, self-contained solution for image acquisition, processing, and display, creating a comprehensive platform for high-speed, reconfigurable image processing.

6. Complete Code with comments:

Python Code

```
"""
Fast UART Image Sender for FPGA Image Processor
Optimized for maximum transmission speed
"""

import cv2
import serial
import numpy as np
import time
import argparse
import os
```

```
import threading
from queue import Queue

class FastUARTImageSender:
    def __init__(self, port, baud_rate=1250000):
        """
        Initialize the fast UART image sender

        Args:
            port (str): Serial port name (e.g., 'COM3' on Windows, '/dev/ttyUSB0' on Linux)
            baud_rate (int): UART baud rate
        """
        self.port = port
        self.baud_rate = baud_rate
        self.serial_conn = None
        self.target_width = 640
        self.target_height = 480

    def connect(self):
        """Connect to the UART port with optimized settings"""
        try:
            self.serial_conn = serial.Serial(
                port=self.port,
                baudrate=self.baud_rate,
                bytesize=serial.EIGHTBITS,
                parity=serial.PARITY_NONE,
                stopbits=serial.STOPBITS_ONE,
                timeout=0.1,        # Reduced timeout for faster operation
                write_timeout=0.1,  # Write timeout to prevent blocking
                # Hardware flow control disabled for maximum speed
                rtscts=False,
                dsrdtr=False
            )

            # Optimize buffer sizes
            self.serial_conn.set_buffer_size(rx_size=4096, tx_size=8192)

            print(f"Connected to {self.port} at {self.baud_rate} baud")
            return True
        except serial.SerialException as e:
            print(f"Failed to connect to {self.port}: {e}")
            return False

    def disconnect(self):
        """Disconnect from the UART port"""
        if self.serial_conn and self.serial_conn.is_open:
            self.serial_conn.close()
            print("Disconnected from serial port")

    def preprocess_image(self, image_path):
        """
        Load and preprocess image for FPGA display (NO ENHANCEMENT)
```

```
    Args:
        image_path (str): Path to image file

    Returns:
        numpy.ndarray: Preprocessed grayscale image
    """
    # Load image
    img = cv2.imread(image_path)
    if img is None:
        raise ValueError(f"Could not load image from {image_path}")

    print(f"Original image shape: {img.shape}")

    # Convert to grayscale
    if len(img.shape) == 3:
        img_gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    else:
        img_gray = img

    # Resize to target resolution using faster interpolation
    img_resized = cv2.resize(img_gray, (self.target_width, self.target_height),
                             interpolation=cv2.INTER_LINEAR)  # Faster than LANCZOS4

    print(f"Processed image shape: {img_resized.shape}")
    return img_resized

def send_image_chunked(self, image, chunk_size=1024):
    """
    Send image in chunks for maximum speed

    Args:
        image (numpy.ndarray): Grayscale image array
        chunk_size (int): Size of chunks to send at once

    Returns:
        bool: True if successful
    """
    if not self.serial_conn or not self.serial_conn.is_open:
        print("Serial connection not established")
        return False

    # Flatten image for transmission
    image_data = image.flatten()
    total_pixels = len(image_data)

    print(f"Starting fast image transmission ({total_pixels} pixels)...")
    print("Progress: [", end="", flush=True)

    start_time = time.time()
    sent_pixels = 0
```

```
        try:
            # Send data in chunks
            for i in range(0, total_pixels, chunk_size):
                chunk = image_data[i:i+chunk_size]

                # Convert numpy array to bytes
                chunk_bytes = chunk.tobytes()

                # Send entire chunk at once
                bytes_written = self.serial_conn.write(chunk_bytes)
                sent_pixels += len(chunk)

                # Force immediate transmission
                self.serial_conn.flush()

                # Show progress
                if sent_pixels % (total_pixels // 50) == 0:
                    print("█", end="", flush=True)

            print("]")
            elapsed_time = time.time() - start_time
            print(f"Transmission completed in {elapsed_time:.2f} seconds")
            print(f"Transfer rate: {total_pixels/elapsed_time:.0f} pixels/second")
            print(f"Data rate: {(total_pixels/elapsed_time)/1000:.1f} KB/s")
            return True

        except serial.SerialException as e:
            print(f"Error during transmission: {e}")
            return False

    def send_image_burst(self, image, burst_size=4096):
        """
        Send image in large bursts for maximum throughput

        Args:
            image (numpy.ndarray): Grayscale image array
            burst_size (int): Size of bursts to send

        Returns:
            bool: True if successful
        """
        if not self.serial_conn or not self.serial_conn.is_open:
            print("Serial connection not established")
            return False

        # Flatten image for transmission
        image_data = image.flatten()
        total_pixels = len(image_data)

        print(f"Starting burst transmission ({total_pixels} pixels)...")
        print("Progress: [", end="", flush=True)
```

```
        start_time = time.time()
        sent_pixels = 0

        try:
            # Send in large bursts
            for i in range(0, total_pixels, burst_size):
                burst = image_data[i:i+burst_size]
                burst_bytes = burst.tobytes()

                # Send burst
                self.serial_conn.write(burst_bytes)
                sent_pixels += len(burst)

                # Show progress less frequently for speed
                if sent_pixels % (total_pixels // 20) == 0:
                    print("", end="", flush=True)

            # Final flush
            self.serial_conn.flush()

            print("]")
            elapsed_time = time.time() - start_time
            print(f"Burst transmission completed in {elapsed_time:.2f} seconds")
            print(f"Transfer rate: {total_pixels/elapsed_time:.0f} pixels/second")
            print(f"Data rate: {(total_pixels/elapsed_time)/1000:.1f} KB/s")
            return True

        except serial.SerialException as e:
            print(f"Error during burst transmission: {e}")
            return False

    def create_test_patterns(self):
        """Create test patterns for demonstrating different processing modes"""
        patterns = {}

        # Gradient test pattern
        gradient = np.zeros((self.target_height, self.target_width), dtype=np.uint8)
        for x in range(self.target_width):
            gradient[:, x] = int(255 * x / self.target_width)
        patterns['gradient'] = gradient

        # Checkerboard pattern
        checkerboard = np.zeros((self.target_height, self.target_width), dtype=np.uint8)
        square_size = 40
        for y in range(0, self.target_height, square_size):
            for x in range(0, self.target_width, square_size):
                if ((x // square_size) + (y // square_size)) % 2 == 0:
                    checkerboard[y:y+square_size, x:x+square_size] = 255
        patterns['checkerboard'] = checkerboard

        # Circular pattern
        center_y, center_x = self.target_height // 2, self.target_width // 2
```

```
        y, x = np.ogrid[:self.target_height, :self.target_width]
        dist = np.sqrt((x - center_x)**2 + (y - center_y)**2)
        circles = ((dist % 50) < 25).astype(np.uint8) * 255
        patterns['circles'] = circles

        # Edge detection test pattern
        edges = np.zeros((self.target_height, self.target_width), dtype=np.uint8)
        # Horizontal edges
        edges[100:110, :] = 255
        edges[200:210, :] = 255
        edges[300:310, :] = 255
        # Vertical edges
        edges[:, 100:110] = 255
        edges[:, 300:310] = 255
        edges[:, 500:510] = 255
        patterns['edges'] = edges

        return patterns

    def save_test_patterns(self, output_dir="test_patterns"):
        """Save test patterns as image files"""
        patterns = self.create_test_patterns()

        os.makedirs(output_dir, exist_ok=True)

        for name, pattern in patterns.items():
            filename = os.path.join(output_dir, f"{name}.png")
            cv2.imwrite(filename, pattern)
            print(f"Saved {filename}")

        return patterns

def main():
    parser = argparse.ArgumentParser(description='Fast UART Image Sender for FPGA')
    parser.add_argument('port', help='Serial port (e.g., COM3, /dev/ttyUSB0)')
    parser.add_argument('--image', '-i', help='Image file to send')
    parser.add_argument('--pattern', '-p',
                        choices=['gradient', 'checkerboard', 'circles', 'edges'],
                        help='Test pattern to send')
    parser.add_argument('--baud', '-b', type=int, default=1250000,
                        help='Baud rate (default: 1250000)')
    parser.add_argument('--mode', '-m', choices=['chunked', 'burst'], default='burst',
                        help='Transmission mode (default: burst)')
    parser.add_argument('--chunk-size', type=int, default=1024,
                        help='Chunk size for chunked mode (default: 1024)')
    parser.add_argument('--burst-size', type=int, default=4096,
                        help='Burst size for burst mode (default: 4096)')
    parser.add_argument('--save-patterns', action='store_true',
                        help='Save test patterns as image files')

    args = parser.parse_args()
```

```
    # Create sender instance
    sender = FastUARTImageSender(args.port, args.baud)

    # Save test patterns if requested
    if args.save_patterns:
        sender.save_test_patterns()
        print("Test patterns saved. You can now send them with --image option.")
        return

    # Connect to UART
    if not sender.connect():
        return

    try:
        if args.image:
            # Send image file
            image = sender.preprocess_image(args.image)
            print(f"Sending image: {args.image}")

            if args.mode == 'chunked':
                sender.send_image_chunked(image, args.chunk_size)
            else:  # burst mode
                sender.send_image_burst(image, args.burst_size)

        elif args.pattern:
            # Send test pattern
            patterns = sender.create_test_patterns()
            if args.pattern in patterns:
                print(f"Sending test pattern: {args.pattern}")

                if args.mode == 'chunked':
                    sender.send_image_chunked(patterns[args.pattern], args.chunk_size)
                else:  # burst mode
                    sender.send_image_burst(patterns[args.pattern], args.burst_size)
            else:
                print(f"Unknown pattern: {args.pattern}")

        else:
            print("Please specify either --image or --pattern")
            print("\nAvailable commands:")
            print("1. Send image (burst mode): python fast_uart_sender.py COM3 --image photo.jpg")
            print("2. Send image (chunked):    python fast_uart_sender.py COM3 --image photo.jpg --mode
chunked")
            print("3. Send test pattern:       python fast_uart_sender.py COM3 --pattern gradient")
            print("4. Custom burst size:       python fast_uart_sender.py COM3 --image photo.jpg --burst-size 8192")
            print("5. Save test patterns:      python fast_uart_sender.py COM3 --save-patterns")
            print("\nSpeed Tips:")
            print("- Use burst mode for maximum speed")
            print("- Increase burst-size for faster transfer (try 8192 or 16384)")
            print("- Ensure good USB connection for high baud rates")
            print("\nFPGA Controls:")
            print("- Switches SW[1:0]: Select processing mode")
```

```
        print("  00 = Black & White, 01 = Grayscale, 10 = Color, 11 = Sobel")
        print("- Center Button: Clear display")
        print("- LEDs show transfer progress and system status")

    finally:
        sender.disconnect()

if __name__ == "__main__":
    main()
```

Verilog Code

Main Module

```
`timescale 1ns/1ps

module top_image_processor_basys3 (
    input wire clk,          // 100 MHz on Basys3 (W5)
    input wire [1:0] sw,     // Mode selection switches
    input wire btn_clear,    // Clear button to reset display
    input wire RsRx,         // UART RX from PC
    output wire RsTx,        // UART TX back to PC (idle high)
    output wire Hsync,
    output wire Vsync,
    output wire [3:0] vgaRed,
    output wire [3:0] vgaGreen,
    output wire [3:0] vgaBlue,
    output wire [15:0] led   // Extended LEDs for mode and progression indication
);

// ====================== Parameters ======================
localparam integer IMG_W = 640;
localparam integer IMG_H = 480;
localparam integer FRAME_PIX = IMG_W * IMG_H; // 307200

// Mode definitions
localparam MODE_BW = 2'b00;         // Black & White
localparam MODE_GRAY = 2'b01;       // Enhanced Grayscale
localparam MODE_COLOR = 2'b10;      // Enhanced Color
localparam MODE_SOBEL = 2'b11;      // Sobel Edge Detection

// ====================== Clock Generation ======================
wire clk_pix; // 25 MHz for VGA
wire clk_sys; // 100 MHz system
wire pll_locked;

clock_gen_basys3 u_clk (
    .clk_in(clk),
    .clk_pix(clk_pix),
    .clk_sys(clk_sys),
    .locked(pll_locked)
);

// ====================== Button Debouncing ======================
```

```
wire btn_clear_debounced;
debounce_onepulse u_btn_debounce (
    .clk(clk_sys),
    .rstn(pll_locked),
    .btn(btn_clear),
    .tick(btn_clear_debounced)
);

// ====================== Mode Selection & LED Indicators ======================
reg [1:0] mode_reg;
reg [1:0] prev_mode_reg;
reg mode_changed;

always @(posedge clk_sys) begin
    if (!pll_locked) begin
        mode_reg <= MODE_BW;
        prev_mode_reg <= MODE_BW;
        mode_changed <= 1'b0;
    end else begin
        prev_mode_reg <= mode_reg;
        mode_reg <= sw;
        mode_changed <= (mode_reg != sw);
    end
end

// ====================== Clear Functionality ======================
reg clear_active;
reg [18:0] clear_addr;
wire clear_complete = (clear_addr >= FRAME_PIX-1);

always @(posedge clk_sys) begin
    if (!pll_locked) begin
        clear_active <= 1'b0;
        clear_addr <= 19'd0;
    end else if (btn_clear_debounced) begin
        clear_active <= 1'b1;
        clear_addr <= 19'd0;
    end else if (clear_active && !clear_complete) begin
        clear_addr <= clear_addr + 1'b1;
    end else if (clear_complete) begin
        clear_active <= 1'b0;
        clear_addr <= 19'd0;
    end
end

// ====================== UART Receiver ======================
wire [7:0] rx_data;
wire rx_valid;

// Use original baud rate for compatibility
uart_rx #(
    .CLK_FREQ_HZ(100_000_000),
```

```
    .BAUD(1_250_000)
) u_rx (
    .clk(clk_sys),
    .rstn(pll_locked),
    .rx(RsRx),
    .data(rx_data),
    .valid(rx_valid)
);

// TX held idle
assign RsTx = 1'b1;

// ====================== Input Pixel Tracking ====================
reg [9:0] x_sys = 10'd0;   // 0..639
reg [9:0] y_sys = 10'd0;   // 0..479
reg [18:0] pix_cnt = 19'd0;
reg transfer_active = 1'b0;

always @(posedge clk_sys) begin
    if (!pll_locked || btn_clear_debounced) begin
        x_sys <= 0;
        y_sys <= 0;
        pix_cnt <= 0;
        transfer_active <= 1'b0;
    end else if (rx_valid) begin
        transfer_active <= 1'b1;
        if (x_sys == IMG_W-1) begin
            x_sys <= 0;
            if (y_sys == IMG_H-1) begin
                y_sys <= 0;
                pix_cnt <= 0;
                transfer_active <= 1'b0; // Frame complete
            end else begin
                y_sys <= y_sys + 1'b1;
                pix_cnt <= pix_cnt + 1'b1;
            end
        end else begin
            x_sys <= x_sys + 1'b1;
            pix_cnt <= pix_cnt + 1'b1;
        end
    end
end

// ====================== Enhanced Processing Pipeline ====================
// Enhanced pixel processing with more color levels and grayscale gradients
reg [7:0] processed_pixel;
reg processed_valid;

always @(posedge clk_sys) begin
    if (!pll_locked) begin
        processed_pixel <= 8'd0;
        processed_valid <= 1'b0;
```

```
end else begin
    processed_valid <= rx_valid;
    if (rx_valid) begin
        case (mode_reg)
            MODE_BW: begin
                // Enhanced Black & White with smoother thresholding
                processed_pixel <= (rx_data >= 8'd128) ? 8'd255 : 8'd0;
            end

            MODE_GRAY: begin
                // Enhanced 8-level grayscale quantization
                if (rx_data >= 8'd224)
                    processed_pixel <= 8'd255;
                else if (rx_data >= 8'd192)
                    processed_pixel <= 8'd224;
                else if (rx_data >= 8'd160)
                    processed_pixel <= 8'd192;
                else if (rx_data >= 8'd128)
                    processed_pixel <= 8'd160;
                else if (rx_data >= 8'd96)
                    processed_pixel <= 8'd128;
                else if (rx_data >= 8'd64)
                    processed_pixel <= 8'd96;
                else if (rx_data >= 8'd32)
                    processed_pixel <= 8'd64;
                else
                    processed_pixel <= 8'd32;
            end

            MODE_COLOR: begin
                // Enhanced false color mapping with more gradients
                case (rx_data[7:4])
                    4'h0: processed_pixel <= 8'd0;    // Black
                    4'h1: processed_pixel <= 8'd32;   // Dark Blue
                    4'h2: processed_pixel <= 8'd64;   // Blue
                    4'h3: processed_pixel <= 8'd96;   // Cyan-Blue
                    4'h4: processed_pixel <= 8'd128;  // Cyan
                    4'h5: processed_pixel <= 8'd160;  // Green-Cyan
                    4'h6: processed_pixel <= 8'd192;  // Green
                    4'h7: processed_pixel <= 8'd224;  // Yellow-Green
                    4'h8: processed_pixel <= 8'd255;  // Yellow
                    4'h9: processed_pixel <= 8'd240;  // Orange-Yellow
                    4'hA: processed_pixel <= 8'd224;  // Orange
                    4'hB: processed_pixel <= 8'd208;  // Red-Orange
                    4'hC: processed_pixel <= 8'd192;  // Red
                    4'hD: processed_pixel <= 8'd176;  // Magenta-Red
                    4'hE: processed_pixel <= 8'd160;  // Magenta
                    4'hF: processed_pixel <= 8'd255;  // White
                endcase
            end

            MODE_SOBEL: begin
```

```
                // Pass through for Sobel - will be processed separately
                processed_pixel <= rx_data;
            end
        endcase
    end
  end
end

// ========== Sobel Edge Detection ==========
wire sobel_valid;
wire sobel_edge;

sobel3x3_stream #(
  .IMG_W(IMG_W)
) u_sobel (
  .clk(clk_sys),
  .rstn(pll_locked),
  .in_valid(rx_valid),
  .in_pix(rx_data),
  .thresh(8'd64),  // Fixed threshold for simplicity
  .out_valid(sobel_valid),
  .out_edge(sobel_edge)
);

// ========== Framebuffer Interface ==========
// Dual-port BRAM for 8-bit pixels (640x480)
localparam FB_ADDR_WIDTH = 19; // log2(640*480)

wire [7:0] fb_wr_data;
wire fb_wr_en;
wire [FB_ADDR_WIDTH-1:0] fb_wr_addr;
wire [FB_ADDR_WIDTH-1:0] fb_rd_addr;
wire [7:0] fb_rd_data;

// Write data selection based on mode and clear operation
assign fb_wr_data = clear_active ? 8'd0 :
                    (mode_reg == MODE_SOBEL) ? (sobel_edge ? 8'd255 : 8'd0) : processed_pixel;
assign fb_wr_en = clear_active || ((mode_reg == MODE_SOBEL) ? sobel_valid : processed_valid);
assign fb_wr_addr = clear_active ? clear_addr : (y_sys * IMG_W) + x_sys;

// Simple dual-port BRAM
(* ram_style = "block" *) reg [7:0] framebuffer [0:FRAME_PIX-1];

// Write port
always @(posedge clk_sys) begin
  if (fb_wr_en) begin
    framebuffer[fb_wr_addr] <= fb_wr_data;
  end
end

// Read port (for VGA)
reg [7:0] fb_rd_data_reg;
```

```
always @(posedge clk_pix) begin
   fb_rd_data_reg <= framebuffer[fb_rd_addr];
end
assign fb_rd_data = fb_rd_data_reg;

// ====================== VGA Controller ======================
wire vga_active;
wire [9:0] vga_x, vga_y;

vga_640x480 u_vga (
   .clk_pix(clk_pix),
   .rstn(pll_locked),
   .hs(Hsync),
   .vs(Vsync),
   .active(vga_active),
   .x(vga_x),
   .y(vga_y)
);

// VGA read address
assign fb_rd_addr = vga_active ? (vga_y * IMG_W + vga_x) : 19'd0;

// ====================== Enhanced VGA Color Output ======================
reg [3:0] vga_r_reg, vga_g_reg, vga_b_reg;

always @(posedge clk_pix) begin
   if (!pll_locked || !vga_active) begin
      vga_r_reg <= 4'd0;
      vga_g_reg <= 4'd0;
      vga_b_reg <= 4'd0;
   end else begin
      case (mode_reg)
         MODE_BW: begin
            // Black and white
            if (fb_rd_data >= 8'd128) begin
               vga_r_reg <= 4'hF;
               vga_g_reg <= 4'hF;
               vga_b_reg <= 4'hF;
            end else begin
               vga_r_reg <= 4'h0;
               vga_g_reg <= 4'h0;
               vga_b_reg <= 4'h0;
            end
         end

         MODE_GRAY: begin
            // Enhanced grayscale with full range
            vga_r_reg <= fb_rd_data[7:4];
            vga_g_reg <= fb_rd_data[7:4];
            vga_b_reg <= fb_rd_data[7:4];
         end
```

```
        MODE_COLOR: begin
          // Enhanced false color mapping with more levels
          case (fb_rd_data[7:4])
            4'h0: begin vga_r_reg <= 4'h0; vga_g_reg <= 4'h0; vga_b_reg <= 4'h0; end // Black
            4'h1: begin vga_r_reg <= 4'h0; vga_g_reg <= 4'h0; vga_b_reg <= 4'h4; end // Dark Blue
            4'h2: begin vga_r_reg <= 4'h0; vga_g_reg <= 4'h0; vga_b_reg <= 4'h8; end // Blue
            4'h3: begin vga_r_reg <= 4'h0; vga_g_reg <= 4'h4; vga_b_reg <= 4'hC; end // Cyan-Blue
            4'h4: begin vga_r_reg <= 4'h0; vga_g_reg <= 4'h8; vga_b_reg <= 4'hC; end // Cyan
            4'h5: begin vga_r_reg <= 4'h0; vga_g_reg <= 4'hC; vga_b_reg <= 4'h8; end // Green-Cyan
            4'h6: begin vga_r_reg <= 4'h0; vga_g_reg <= 4'hC; vga_b_reg <= 4'h0; end // Green
            4'h7: begin vga_r_reg <= 4'h4; vga_g_reg <= 4'hC; vga_b_reg <= 4'h0; end // Yellow-Green
            4'h8: begin vga_r_reg <= 4'h8; vga_g_reg <= 4'hC; vga_b_reg <= 4'h0; end // Yellow
            4'h9: begin vga_r_reg <= 4'hC; vga_g_reg <= 4'hC; vga_b_reg <= 4'h0; end // Orange-Yellow
            4'hA: begin vga_r_reg <= 4'hC; vga_g_reg <= 4'h8; vga_b_reg <= 4'h0; end // Orange
            4'hB: begin vga_r_reg <= 4'hC; vga_g_reg <= 4'h4; vga_b_reg <= 4'h0; end // Red-Orange
            4'hC: begin vga_r_reg <= 4'hC; vga_g_reg <= 4'h0; vga_b_reg <= 4'h0; end // Red
            4'hD: begin vga_r_reg <= 4'hC; vga_g_reg <= 4'h0; vga_b_reg <= 4'h4; end // Magenta-Red
            4'hE: begin vga_r_reg <= 4'hC; vga_g_reg <= 4'h0; vga_b_reg <= 4'h8; end // Magenta
            4'hF: begin vga_r_reg <= 4'hF; vga_g_reg <= 4'hF; vga_b_reg <= 4'hF; end // White
          endcase
        end

        MODE_SOBEL: begin
          // Sobel edges in white on black
          if (fb_rd_data >= 8'd128) begin
            vga_r_reg <= 4'hF;
            vga_g_reg <= 4'hF;
            vga_b_reg <= 4'hF;
          end else begin
            vga_r_reg <= 4'h0;
            vga_g_reg <= 4'h0;
            vga_b_reg <= 4'h0;
          end
        end
      endcase
    end
end

assign vgaRed = vga_r_reg;
assign vgaGreen = vga_g_reg;
assign vgaBlue = vga_b_reg;

// ====================== LED Status Indicators ======================
reg [15:0] led_reg;
reg [23:0] blink_counter;
wire blink_slow = blink_counter[23];     // ~6 Hz blink
wire blink_fast = blink_counter[20];     // ~50 Hz blink

always @(posedge clk_sys) begin
    if (!pll_locked) begin
        blink_counter <= 24'd0;
    end else begin
```

```
        blink_counter <= blink_counter + 1'b1;
    end
end

// Progress calculation for LEDs (avoid division by zero)
wire [7:0] progress_percent = (FRAME_PIX > 0 && pix_cnt > 0) ?
    ((pix_cnt < FRAME_PIX) ? ((pix_cnt * 100) / FRAME_PIX) : 8'd100) : 8'd0;

wire [3:0] progress_leds = (progress_percent >= 8'd90) ? 4'hF :
                           (progress_percent >= 8'd70) ? 4'h7 :
                           (progress_percent >= 8'd50) ? 4'h3 :
                           (progress_percent >= 8'd25) ? 4'h1 : 4'h0;

always @(posedge clk_sys) begin
    if (!pll_locked) begin
        led_reg <= 16'd0;
    end else begin
        // LEDs 0-3: Mode indicators (one-hot)
        led_reg[0] <= (mode_reg == MODE_BW);
        led_reg[1] <= (mode_reg == MODE_GRAY);
        led_reg[2] <= (mode_reg == MODE_COLOR);
        led_reg[3] <= (mode_reg == MODE_SOBEL);

        // LEDs 4-7: Transfer progress
        if (transfer_active) begin
            led_reg[7:4] <= progress_leds;
        end else begin
            led_reg[7:4] <= 4'h0;
        end

        // LEDs 8-11: Status indicators
        led_reg[8] <= transfer_active ? blink_fast : 1'b0;  // Transfer active (fast blink)
        led_reg[9] <= clear_active ? blink_fast : 1'b0;     // Clear active (fast blink)
        led_reg[10] <= mode_changed ? blink_slow : 1'b0;    // Mode changed (slow blink)
        led_reg[11] <= pll_locked;                          // System ready

        // LEDs 12-15: Additional status
        led_reg[12] <= rx_valid;                            // UART activity
        led_reg[13] <= (pix_cnt > 19'd0);                   // Frame data present
        led_reg[14] <= (y_sys == IMG_H-1 && x_sys == IMG_W-1); // Frame complete
        led_reg[15] <= vga_active;                          // VGA active
    end
end

assign led = led_reg;

endmodule
```

Clock Generation

```
`timescale 1ns/1ps
// 100 MHz in (Basys3 W5) -> 25.000 MHz pixel clock out + pass-through 100 MHz
module clock_gen_basys3 (
    input  wire clk_in,   // 100 MHz
    output wire clk_pix,  // 25 MHz
    output wire clk_sys,  // 100 MHz (buffered)
    output wire locked
);
    // Input buffer
    wire clk_in_ibuf;
    IBUF ibuf_clk (.I(clk_in), .O(clk_in_ibuf));

    // Route 100 MHz to a global clock buffer (optional but good practice)
    BUFG bufg_sys (.I(clk_in_ibuf), .O(clk_sys));

    // MMCM signals
    wire clkfb, clkfb_buf;
    wire clkout0_mmcm; // 25 MHz before BUFG

    // VCO = 100 MHz * 10 = 1000 MHz (in 600..1200 range), CLKOUT0 = 1000/40 = 25 MHz
    MMCME2_BASE #(
        .BANDWIDTH("OPTIMIZED"),
        .CLKIN1_PERIOD(10.0),   // 100 MHz
        .DIVCLK_DIVIDE(1),
        .CLKFBOUT_MULT_F(10.0), // VCO = 100*10
        .CLKFBOUT_PHASE(0.0),

        .CLKOUT0_DIVIDE_F(40.0),// 1000/40 = 25.000 MHz
        .CLKOUT0_PHASE(0.0),
        .CLKOUT0_DUTY_CYCLE(0.5),

        // Unused outputs default
        .CLKOUT1_DIVIDE(1), .CLKOUT2_DIVIDE(1), .CLKOUT3_DIVIDE(1),
        .CLKOUT4_DIVIDE(1), .CLKOUT5_DIVIDE(1), .CLKOUT6_DIVIDE(1),
        .STARTUP_WAIT("FALSE")
    ) u_mmcm (
        .CLKIN1   (clk_in_ibuf),
        .CLKFBIN  (clkfb_buf),
        .CLKFBOUT (clkfb),
        .CLKOUT0  (clkout0_mmcm),

        .LOCKED   (locked),
        .PWRDWN   (1'b0),
        .RST      (1'b0)
    );

    // Feedback path must use a BUFG
    BUFG clkf_buf (.I(clkfb), .O(clkfb_buf));

    // Buffer the 25 MHz pixel clock to a global clock net
    BUFG bufg_pix (.I(clkout0_mmcm), .O(clk_pix));
```

Endmodule

Debounce (Buttons)

```
`timescale 1ns/1ps
module debounce_onepulse (
input wire clk, // 100 MHz
input wire rstn,
input wire btn,
output wire tick
);
// 2-flop sync
reg s0,s1; always @(posedge clk) begin if(!rstn) begin s0<=0; s1<=0; end else begin s0<=btn; s1<=s0; end end

// Debounce ~5ms
reg [18:0] cnt; reg db;
always @(posedge clk) begin
if(!rstn) begin cnt<=0; db<=0; end
else if(s1==db) cnt<=0;
else if(cnt==19'd499_999) begin db<=s1; cnt<=0; end
else cnt<=cnt+1'b1;
end
reg db_d; always @(posedge clk) begin if(!rstn) db_d<=0; else db_d<=db; end
assign tick = db & ~db_d;
endmodule
```

UART Rx

```
`timescale 1ns/1ps
module uart_rx #(
parameter integer CLK_FREQ_HZ = 100_000_000,
parameter integer BAUD = 1_250_000 // integer 16x: DIV = 5 at 100 MHz
)(
input wire clk,
input wire rstn,
input wire rx,
output reg [7:0] data,
output reg valid
);
localparam integer DIV = CLK_FREQ_HZ/(BAUD*16);
// protect against bad params
initial begin if (DIV<1) $error("UART RX: DIV < 1"); end

// 16x baud tick
reg [$clog2(DIV)-1:0] bd_cnt=0; reg baud16x=0;
always @(posedge clk) begin
if(!rstn) begin bd_cnt<=0; baud16x<=0; end
else if(bd_cnt==DIV-1) begin bd_cnt<=0; baud16x<=1; end
else begin bd_cnt<=bd_cnt+1'b1; baud16x<=0; end
end
// Sync RX
reg r0,r1; always @(posedge clk) begin r0<=rx; r1<=r0; end
```

```
// FSM
localparam [1:0] IDLE=0, START=1, DATA=2, STOP=3;
reg [1:0] st=IDLE; reg [3:0] sc=0; reg [2:0] bc=0; reg [7:0] sh=0;

always @(posedge clk) begin
if(!rstn) begin st<=IDLE; sc<=0; bc<=0; sh<=0; data<=0; valid<=0; end
else begin
valid<=0;
if(baud16x) begin
case(st)
IDLE: if(!r1) begin st<=START; sc<=0; end
START: if(sc==7) begin if(!r1) begin st<=DATA; sc<=0; bc<=0; end else st<=IDLE; end else sc<=sc+1;
DATA: if(sc==15) begin sh<={r1,sh[7:1]}; sc<=0; if(bc==7) st<=STOP; else bc<=bc+1; end else sc<=sc+1;
STOP: if(sc==15) begin data<=sh; valid<=1; st<=IDLE; sc<=0; end else sc<=sc+1;
                                        endcase
                                    end
                                end
                            end
endmodule
```

Sobel Filter

```
`timescale 1ns/1ps
module sobel3x3_stream #(
parameter integer IMG_W = 640
)(
input wire clk,
input wire rstn,
input wire in_valid,
input wire [7:0] in_pix,
input wire [7:0] thresh,
output reg out_valid,
output reg out_edge
);
// Two previous rows in BRAM-like arrays
(* ram_style = "block" *) reg [7:0] line0 [0:IMG_W-1];
(* ram_style = "block" *) reg [7:0] line1 [0:IMG_W-1];

reg [9:0] x = 10'd0; // 0..639
reg primed = 1'b0; // after >= 2 rows

// 3x3 window regs
reg [7:0] p00,p01,p02;
reg [7:0] p10,p11,p12;
reg [7:0] p20,p21,p22;

// pipeline valids
reg v1, v2;


// arithmetic temps
```

```
integer gx, gy, mag;

always @(posedge clk) begin
if(!rstn) begin
x<=0; primed<=1'b0; v1<=0; v2<=0; out_valid<=0; out_edge<=0;
p00<=0;p01<=0;p02<=0; p10<=0;p11<=0;p12<=0; p20<=0;p21<=0;p22<=0;
end else begin
v1 <= in_valid;
v2 <= v1;
out_valid <= 1'b0;

if (in_valid) begin
// shift window
p00<=p01; p01<=p02; p02<= line1[x];
p10<=p11; p11<=p12; p12<= line0[x];
p20<=p21; p21<=p22; p22<= in_pix;

// update lines
line1[x] <= line0[x];
line0[x] <= in_pix;

// column advance
if (x==IMG_W-1) begin
x <= 0;
if (!primed) primed <= 1'b1; // after first wrap, we have 2 lines filled later
end else begin
x <= x + 1'b1;
end
end

if (v2 && primed && x>1) begin
// Sobel
gx = -$signed({1'b0,p00}) - (2*$signed({1'b0,p10})) - $signed({1'b0,p20})
+ $signed({1'b0,p02}) + (2*$signed({1'b0,p12})) + $signed({1'b0,p22});
gy = -$signed({1'b0,p00}) - (2*$signed({1'b0,p01})) - $signed({1'b0,p02})
+ $signed({1'b0,p20}) + (2*$signed({1'b0,p21})) + $signed({1'b0,p22});
if (gx<0) gx=-gx; if (gy<0) gy=-gy;
mag = gx + gy; if (mag>255) mag=255;
out_edge <= (mag[7:0] >= thresh);
out_valid <= 1'b1;
end
end
end
endmodule
```

VGA Output

```
`timescale 1ns/1ps
module vga_640x480 (
input wire clk_pix, // 25 MHz
input wire rstn,
output reg hs,
```

```
output reg vs,
output wire active,
output reg [9:0] x,
output reg [9:0] y
);
localparam H_VISIBLE=640, H_FP=16, H_SYNC=96, H_BP=48, H_TOTAL=800;
localparam V_VISIBLE=480, V_FP=10, V_SYNC=2, V_BP=33, V_TOTAL=525;
reg [9:0] h=0,v=0;
assign active = (h < H_VISIBLE) && (v < V_VISIBLE);

always @(posedge clk_pix) begin
if(!rstn) begin h<=0; v<=0; hs<=1; vs<=1; x<=0; y<=0; end
else begin
// counters
if (h==H_TOTAL-1) begin h<=0; if(v==V_TOTAL-1) v<=0; else v<=v+1; end
else h<=h+1;
// syncs active-low
hs <= ~((h >= H_VISIBLE+H_FP) && (h < H_VISIBLE+H_FP+H_SYNC));
vs <= ~((v >= V_VISIBLE+V_FP) && (v < V_VISIBLE+V_FP+V_SYNC));
// coords
x <= (h < H_VISIBLE) ? h : 10'd0;
y <= (v < V_VISIBLE) ? v : 10'd0;
end
end
endmodule
```

Constraints

```
## Clock signal
set_property PACKAGE_PIN W5 [get_ports clk]
   set_property IOSTANDARD LVCMOS33 [get_ports clk]
   create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports clk]

## Switches for mode selection
set_property PACKAGE_PIN V17 [get_ports {sw[0]}]
   set_property IOSTANDARD LVCMOS33 [get_ports {sw[0]}]
set_property PACKAGE_PIN V16 [get_ports {sw[1]}]
   set_property IOSTANDARD LVCMOS33 [get_ports {sw[1]}]

## Clear button (Center button)
set_property PACKAGE_PIN U18 [get_ports btn_clear]
   set_property IOSTANDARD LVCMOS33 [get_ports btn_clear]

## Extended LEDs for mode indication and progress display (16 LEDs)
# Mode indicators (LEDs 0-3)
set_property PACKAGE_PIN U16 [get_ports {led[0]}]
   set_property IOSTANDARD LVCMOS33 [get_ports {led[0]}]
set_property PACKAGE_PIN E19 [get_ports {led[1]}]
   set_property IOSTANDARD LVCMOS33 [get_ports {led[1]}]
set_property PACKAGE_PIN U19 [get_ports {led[2]}]
   set_property IOSTANDARD LVCMOS33 [get_ports {led[2]}]
set_property PACKAGE_PIN V19 [get_ports {led[3]}]
   set_property IOSTANDARD LVCMOS33 [get_ports {led[3]}]
```

```
# Progress indicators (LEDs 4-7)
set_property PACKAGE_PIN W18 [get_ports {led[4]}]
   set_property IOSTANDARD LVCMOS33 [get_ports {led[4]}]
set_property PACKAGE_PIN U15 [get_ports {led[5]}]
   set_property IOSTANDARD LVCMOS33 [get_ports {led[5]}]
set_property PACKAGE_PIN U14 [get_ports {led[6]}]
   set_property IOSTANDARD LVCMOS33 [get_ports {led[6]}]
set_property PACKAGE_PIN V14 [get_ports {led[7]}]
   set_property IOSTANDARD LVCMOS33 [get_ports {led[7]}]

# Status indicators (LEDs 8-11)
set_property PACKAGE_PIN V13 [get_ports {led[8]}]
   set_property IOSTANDARD LVCMOS33 [get_ports {led[8]}]
set_property PACKAGE_PIN V3 [get_ports {led[9]}]
   set_property IOSTANDARD LVCMOS33 [get_ports {led[9]}]
set_property PACKAGE_PIN W3 [get_ports {led[10]}]
   set_property IOSTANDARD LVCMOS33 [get_ports {led[10]}]
set_property PACKAGE_PIN U3 [get_ports {led[11]}]
   set_property IOSTANDARD LVCMOS33 [get_ports {led[11]}]

# Additional status indicators (LEDs 12-15)
set_property PACKAGE_PIN P3 [get_ports {led[12]}]
   set_property IOSTANDARD LVCMOS33 [get_ports {led[12]}]
set_property PACKAGE_PIN N3 [get_ports {led[13]}]
   set_property IOSTANDARD LVCMOS33 [get_ports {led[13]}]
set_property PACKAGE_PIN P1 [get_ports {led[14]}]
   set_property IOSTANDARD LVCMOS33 [get_ports {led[14]}]
set_property PACKAGE_PIN L1 [get_ports {led[15]}]
   set_property IOSTANDARD LVCMOS33 [get_ports {led[15]}]

## VGA Connector
set_property PACKAGE_PIN G19 [get_ports {vgaRed[0]}]
   set_property IOSTANDARD LVCMOS33 [get_ports {vgaRed[0]}]
set_property PACKAGE_PIN H19 [get_ports {vgaRed[1]}]
   set_property IOSTANDARD LVCMOS33 [get_ports {vgaRed[1]}]
set_property PACKAGE_PIN J19 [get_ports {vgaRed[2]}]
   set_property IOSTANDARD LVCMOS33 [get_ports {vgaRed[2]}]
set_property PACKAGE_PIN N19 [get_ports {vgaRed[3]}]
   set_property IOSTANDARD LVCMOS33 [get_ports {vgaRed[3]}]
set_property PACKAGE_PIN N18 [get_ports {vgaBlue[0]}]
   set_property IOSTANDARD LVCMOS33 [get_ports {vgaBlue[0]}]
set_property PACKAGE_PIN L18 [get_ports {vgaBlue[1]}]
   set_property IOSTANDARD LVCMOS33 [get_ports {vgaBlue[1]}]
set_property PACKAGE_PIN K18 [get_ports {vgaBlue[2]}]
   set_property IOSTANDARD LVCMOS33 [get_ports {vgaBlue[2]}]
set_property PACKAGE_PIN J18 [get_ports {vgaBlue[3]}]
   set_property IOSTANDARD LVCMOS33 [get_ports {vgaBlue[3]}]
set_property PACKAGE_PIN J17 [get_ports {vgaGreen[0]}]
   set_property IOSTANDARD LVCMOS33 [get_ports {vgaGreen[0]}]
set_property PACKAGE_PIN H17 [get_ports {vgaGreen[1]}]
   set_property IOSTANDARD LVCMOS33 [get_ports {vgaGreen[1]}]
```

```
set_property PACKAGE_PIN G17 [get_ports {vgaGreen[2]}]
   set_property IOSTANDARD LVCMOS33 [get_ports {vgaGreen[2]}]
set_property PACKAGE_PIN D17 [get_ports {vgaGreen[3]}]
   set_property IOSTANDARD LVCMOS33 [get_ports {vgaGreen[3]}]
set_property PACKAGE_PIN P19 [get_ports Hsync]
   set_property IOSTANDARD LVCMOS33 [get_ports Hsync]
set_property PACKAGE_PIN R19 [get_ports Vsync]
   set_property IOSTANDARD LVCMOS33 [get_ports Vsync]

## USB-RS232 Interface
set_property PACKAGE_PIN B18 [get_ports RsRx]
   set_property IOSTANDARD LVCMOS33 [get_ports RsRx]
set_property PACKAGE_PIN A18 [get_ports RsTx]
   set_property IOSTANDARD LVCMOS33 [get_ports RsTx]

## Configuration options
set_property CONFIG_VOLTAGE 3.3 [current_design]
set_property CFGBVS VCCO [current_design]

## SPI configuration mode options
set_property BITSTREAM.GENERAL.COMPRESS TRUE [current_design]
set_property BITSTREAM.CONFIG.CONFIGRATE 33 [current_design]
set_property CONFIG_MODE SPIx4 [current_design]

## Timing constraints
set_false_path -from [get_clocks sys_clk_pin] -to [get_clocks -of_objects [get_pins u_clk/clk_pix]]
set_false_path -from [get_clocks -of_objects [get_pins u_clk/clk_pix]] -to [get_clocks sys_clk_pin]

## Button timing constraints (debouncing)
set_false_path -from [get_ports btn_clear] -to [all_clocks]
```

7. RTL:

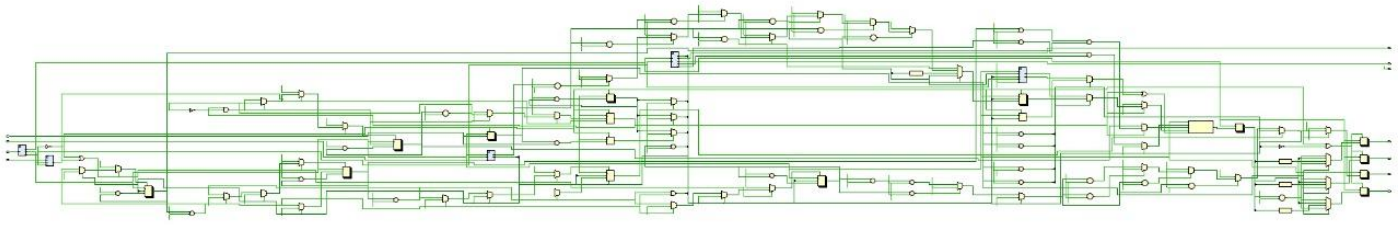


Fig.1. Schematic Diagram

8. Resource Utilization summary table:

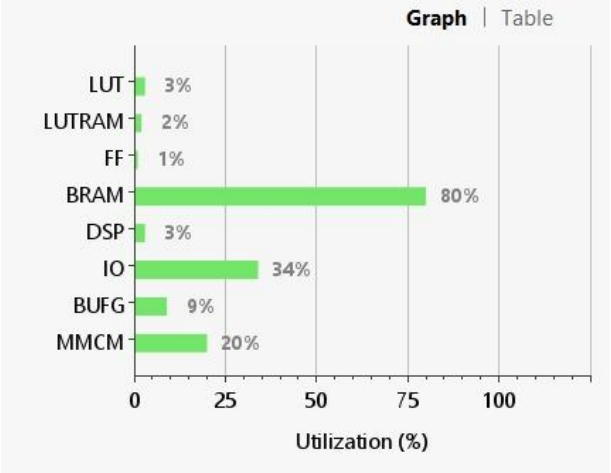


Fig.2. Utilization Graph

Resource	Utilization	Available	Utilization...
LUT	715	20800	3.44
LUTRAM	160	9600	1.67
FF	291	41600	0.70
BRAM	40	50	80.00
DSP	3	90	3.33
IO	36	106	33.96
BUFG	3	32	9.38
MMCM	1	5	20.00

Fig.3. Utilization Table

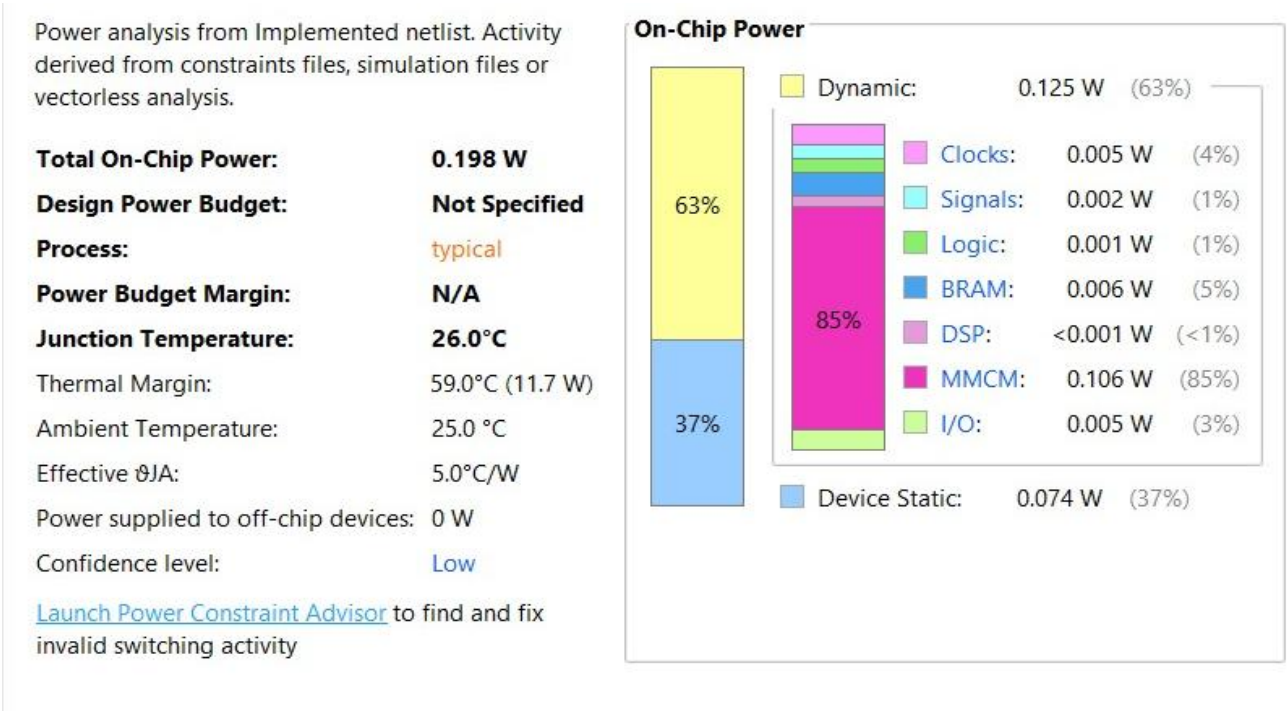


Fig.4. Area-Power Report

9. Output Waveform

```
PS C:\Users\Deenadhayalan\OneDrive\Desktop\sobeledge\Trail_> python fast_uart_sender.py COM20 --image input3.jpg
Connected to COM20 at 1250000 baud
Original image shape: (960, 1440, 3)
Processed image shape: (480, 640)
Sending image: input3.jpg
Starting burst transmission (307200 pixels)...
Progress: [          ]
Burst transmission completed in 2.43 seconds
Transfer rate: 126454 pixels/second
Data rate: 126.5 KB/s
Disconnected from serial port
```

Fig.5. Console Output

10. Hardware Images (if any):

~ **Connections:**

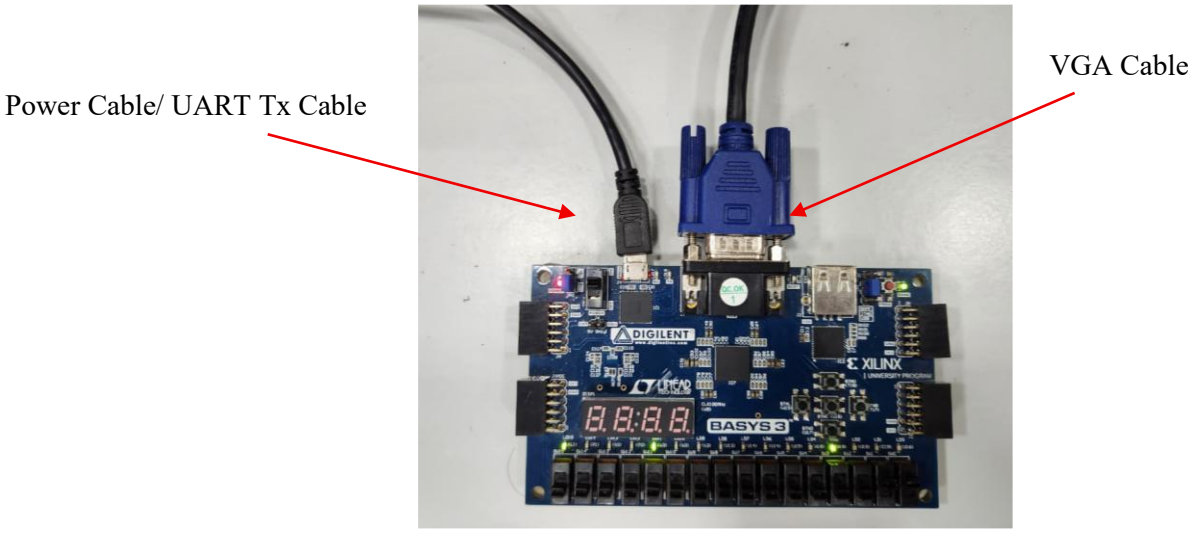


Fig.6. Hardware Connections to FPGA

~ **LED Status:**

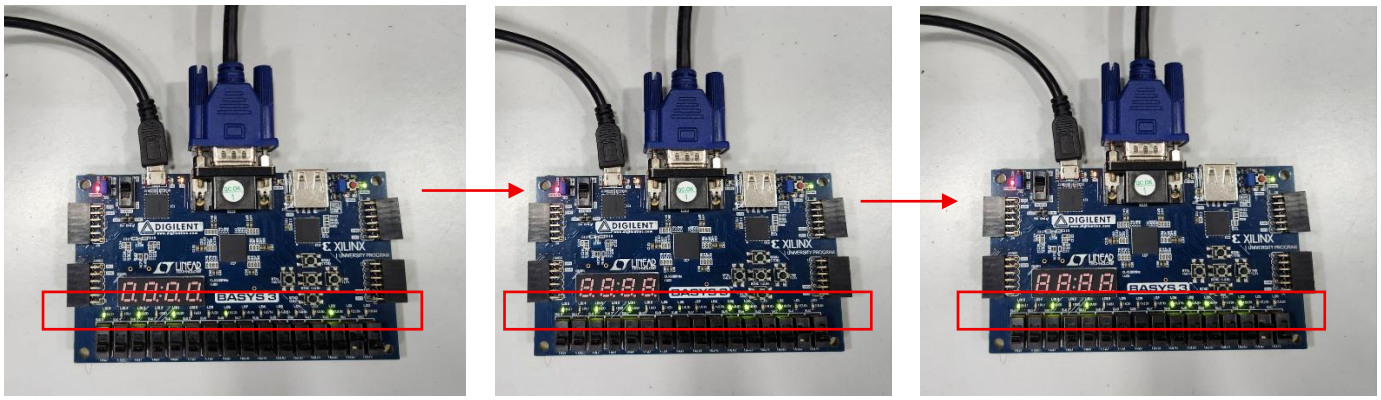


Fig.7. LED Indicators

~ **Output Images for different modes:**

Fig.8. Original Image

Fig.9. B&W Image

Fig.10. Grayscale Image

Fig.11. Color Image

Fig.12. Sober Filter Image